

1. Introduction. A *twelve-tone row* is a permutation of the twelve pitch classes, and Schoenberg's students built whole compositions out of one. Exercise 103 of §7.2.2.1 asks for the rows whose eleven adjacent intervals $(x_k - x_{k-1}) \bmod n$ are all different—the *all-interval* rows—and exercise 104 goes after a rarer breed still.

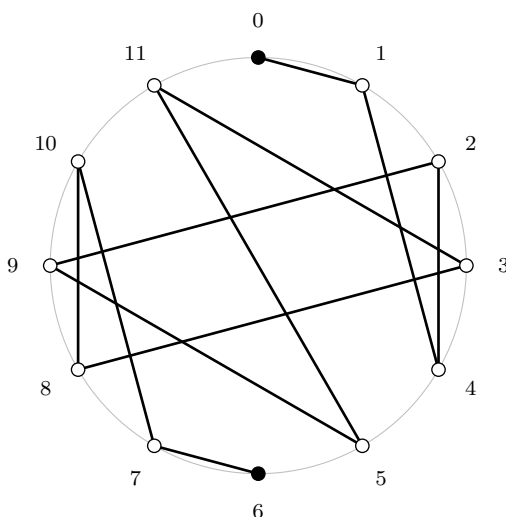
104. [M28] Assume that $n + 1 = p$ is prime. Given an n -tone row $x = x_0x_1 \dots x_{n-1}$, define $y_k = x_{(k-1) \bmod p}$, and let $x^{(r)} = y_ry_{2r} \dots y_{nr}$ be the n -tone row consisting of “every r th element of x ” (where x_n is considered to be blank). For example, when $n = 12$, every 5th element of x is the sequence $x^{(5)} = x_4x_9x_1x_6x_{11}x_3x_8x_0x_5x_{10}x_2x_7$.

An n -tone row is called *perfect* if it is equivalent to $x^{(r)}$ for $1 \leq r \leq n$. For example, the amazing 12-tone row 0 1 4 2 9 5 11 3 8 10 7 6 is perfect.

- Prove that a perfect n -tone row has the all-interval property.
- Prove that a perfect n -tone row also satisfies $x \equiv x^R$.

Two rows are *equivalent* when they differ by a transposition, that is, when $x'_k = (x_k + d) \bmod n$ for some fixed d ; and x^R is the reversal $x_{n-1} \dots x_1x_0$. I have quoted the statement as amended on 15 September 2025; the first printing said “whenever k is not a multiple of p ” in place of the parenthesis about x_n .

2. Here is the amazing row itself, read as a path on the twelve pitch classes. It starts at 0 and finishes at 6, which answer 103(a) says every all-interval row must do; the eleven steps between are eleven different intervals, which is what part (a) of this exercise asks one to prove.



3. I wrote this program in September 2026 to check the exercise and its answer. Everything in both turned out to be right, so what follows is corroboration rather than correction: the printed sequence $x^{(5)}$ is right index for index, the amazing row really is perfect, and both parts of the answer hold for every row that a machine can reach.

The answer to part (a) does more than it advertises. Its argument shows that a perfect row *is* a table of discrete logarithms—that x_{k-1} is the index of k with respect to some primitive root R modulo p —and the converse is easy, so the perfect rows are exactly those tables. There are $\varphi(n)$ of them for each n , one per primitive root, and the amazing row of the exercise is the one belonging to the smallest primitive root of 13.

These notes are the companion document `README.md` in the directory above.

4. The program has four things to say, and a flag chooses which.

```

package main
import (
    "flag"
    "fmt"
    "log"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Functions 6 ⟩
func main() {
    ⟨ Read the command line 5 ⟩
    switch *mode {
    case "example":
        ⟨ Check what the exercise prints 12 ⟩
    case "perfect":
        ⟨ Exhibit the perfect rows 16 ⟩
    case "all":
        ⟨ Search every row exhaustively 19 ⟩
    case "xcc":
        ⟨ Count all-interval rows with dancing cells 22 ⟩
    default:
        log.Fatalf("unknown_mode_%q", *mode)
    }
}

```

5. Twelve is the interesting size and the only one the exercise names, but nothing here is special to it, so the sizes to try are a flag too.

⟨ Read the command line 5 ⟩ ≡

```

mode := flag.String("mode", "example", "example, perfect, all, or xcc")
upto := flag.Int("upto", 12, "largest_n_to_try")
flag.Parse()

```

This code is used in section 4.

6. Rows and their transformations. A row is a slice of the integers 0 through $n - 1$, in some order. Everything the exercise defines is a few lines apiece, but they are used from several places, so they are functions.

The first is the subject of the exercise. Every index arrives as jr for some $1 \leq j \leq n$, and y_k is $x_{(k-1) \bmod p}$, so the j th element of $x^{(r)}$ is $x_{(jr-1) \bmod p}$. Since p is prime and neither j nor r is a multiple of it, jr never is either—which is exactly why the exercise assumes p prime, and why x_n is never wanted.

⟨Functions 6⟩ $\equiv$

```

func sub( $x$  []int,  $r$  int) []int {
     $n := \text{len}(x)$ 
     $p := n + 1$ 
     $out := \text{make}([]\text{int}, n)$ 
    for  $j := 1; j \leq n; j++$  {
         $out[j - 1] = x[(j * r - 1) \% p]$ 
    }
    return out
}

```

This code is also defined in sections 7, 8, 9, 10, 11, 14.

This code is used in section 4.

7. Equivalence is transposition, so two rows are equivalent when the difference of their first elements already accounts for all the rest.

⟨Functions 6⟩ $+ \equiv$

```

func equiv( $a, b$  []int) bool {
     $n := \text{len}(a)$ 
     $d := ((b[0] - a[0]) \% n + n) \% n$ 
    for  $i := \text{range } a$  {
        if  $(a[i] + d) \% n \neq b[i]$  {
            return false
        }
    }
    return true
}

func reversed( $x$  []int) []int {
     $n := \text{len}(x)$ 
     $out := \text{make}([]\text{int}, n)$ 
    for  $i := \text{range } x$  {
         $out[i] = x[n - 1 - i]$ 
    }
    return out
}

```

8. A row is perfect when every $x^{(r)}$ is equivalent to it, and it has the all-interval property when its $n - 1$ adjacent differences are all different. Since a row is a permutation no difference can be zero, so being different is the whole of it.

⟨ Functions 6 ⟩ +≡

```

func perfect(x []int) bool {
  for r := 1; r ≤ len(x); r++ {
    if ¬equiv(x, sub(x, r)) {
      return false
    }
  }
  return true
}

func allInterval(x []int) bool {
  n := len(x)
  seen := make([]bool, n)
  for k := 1; k < n; k++ {
    d := ((x[k] - x[k - 1]) % n + n) % n
    if d ≡ 0 ∨ seen[d] {
      return false
    }
    seen[d] = true
  }
  return true
}

```

9. Three scraps of number theory. The multiplicative order of r modulo p is what tells a primitive root from an impostor, and Euler's φ is the count that the perfect rows will turn out to match.

⟨Functions 6⟩ +≡

```

func isPrime(m int) bool {
  if m < 2 {
    return false
  }
  for d := 2; d * d ≤ m; d++ {
    if m % d ≡ 0 {
      return false
    }
  }
  return true
}

func gcd(a, b int) int {
  for b ≠ 0 {
    a, b = b, a % b
  }
  return a
}

func totient(n int) int {
  c := 0
  for k := 1; k ≤ n; k++ {
    if gcd(k, n) ≡ 1 {
      c++
    }
  }
  return c
}

func order(r, p int) int {
  e, w := 1, r % p
  for w ≠ 1 {
    w, e = w * r % p, e + 1
  }
  return e
}

```

10. Here is the row that the answer to part (a) secretly describes. Walking the powers of R and recording where each residue turns up gives the table of indices $x_{k-1} = \text{ind}_R k$; if R is not a primitive root the walk closes early and there is no row to report.

(Functions 6) +≡

```

func dlogRow( $R, p$  int) []int {
   $n := p - 1$ 
  if order( $R, p$ )  $\neq n$  {
    return  $\Lambda$ 
  }
   $x := \text{make}([]\text{int}, n)$ 
   $v := 1$ 
  for  $e := 0; e < n; e++$  {
     $x[v - 1] = e$ 
     $v = v * R \% p$ 
  }
  return  $x$ 
}

```

11. Finally, a way to walk every row with $x_0 = 0$ —one representative of each equivalence class, since a transposition is settled by its first element. There are $(n - 1)!$ of them, which is 39,916,800 when $n = 12$: large, but not too large to visit one at a time.

(Functions 6) +≡

```

func rows( $n$  int, visit func([]int)) {
   $x := \text{make}([]\text{int}, n)$ 
   $used := \text{make}([]\text{bool}, n)$ 
   $used[0] = \text{true}$ 
  var rec func(int)
   $rec = \text{func}(i$  int) {
    if  $i \equiv n$  {
      visit( $x$ )
      return
    }
    for  $v := 1; v < n; v++$  {
      if  $\neg used[v]$  {
         $used[v], x[i] = \text{true}, v$ 
        rec( $i + 1$ )
         $used[v] = \text{false}$ 
      }
    }
  }
  rec(1)
}

```

12. What the exercise prints. The statement makes two concrete claims, and both can be checked on sight. The first is the sequence of indices for $n = 12$ and $r = 5$.

⟨ Check what the exercise prints 12 ⟩ $\equiv$

```
fmt.Println("the exercise says that every 5th element of a 12-tone row is")
fmt.Println("x4x9x1x6x11x3x8x0x5x10x2x7")
want := []int{4, 9, 1, 6, 11, 3, 8, 0, 5, 10, 2, 7}
got := make([]int, 12)
for j := 1; j ≤ 12; j++ {
    got[j - 1] = (j * 5 - 1) % 13
}
fmt.Printf("the definition gives %v\n", got)
fmt.Printf("agrees: %v\n\n", equiv2(want, got))
```

This code is also defined in sections 13, 15.

This code is used in section 4.

13. The second claim is that one particular row is perfect. It is, and it is more than that: it is the table of indices for $R = 2$, the smallest primitive root of 13.

⟨ Check what the exercise prints 12 ⟩ $+ \equiv$

```
amazing := []int{0, 1, 4, 2, 9, 5, 11, 3, 8, 10, 7, 6}
fmt.Printf("the amazing row %v\n", amazing)
fmt.Printf("is perfect: %v\n", perfect(amazing))
fmt.Printf("has all intervals: %v\n", allInterval(amazing))
fmt.Printf("is equivalent to x^R: %v\n", equiv(amazing, reversed(amazing)))
fmt.Printf("is the index table of R=2: %v\n",
    equiv2(amazing, dlogRow(2, 13)))
```

14. Comparing two slices for plain equality, rather than for equivalence, is wanted twice above and once again later, so it is a function of its own.

⟨ Functions 6 ⟩ $+ \equiv$

```
func equiv2(a, b []int) bool {
    if len(a) ≠ len(b) {
        return false
    }
    for i := range a {
        if a[i] ≠ b[i] {
            return false
        }
    }
    return true
}
```

15. Part (b) of the answer is a single equation, $x^{(n)} = x^R$, and it deserves a word because it is stronger than the exercise needs. Since $n \equiv -1$ modulo p we have $(jn - 1) \bmod p = n - j$, so the j th element of $x^{(n)}$ is x_{n-j} : the reversal exactly, not merely something equivalent to it. It holds for *every* row, perfect or not, and a perfect row is equivalent to $x^{(n)}$ by definition, which finishes part (b).

⟨ Check what the exercise prints 12 ⟩ $+ \equiv$

```

fmt.Println()
bad := 0
for n := 2; n ≤ 12; n++ {
    if ¬isPrime(n + 1) {
        continue
    }
    rows(n, func(x []int) {
        if ¬equiv2(sub(x, n), reversed(x)) {
            bad++
        }
    })
    fmt.Printf("n=%2d: x^(n) = x^R for every row, exceptions so far: %d\n", n, bad)
}

```


16. Perfect rows are tables of logarithms. The answer to part (a) assumes x perfect, picks a primitive root r , and finds a constant c_r with $y_{kr} \equiv x_{k-1} + c_r$. Iterating gives $x_{(r^k-1) \bmod p} \equiv kc_r$, and then it sets $R = r^d$ where $c_r d \bmod n = 1$ to conclude that $R^{x_{k-1}} \equiv k$.

That last move needs d to exist, which needs c_r to be invertible modulo n —and the answer does not stop to say so. It is true, and the reason is the sentence just before it: because r is primitive, $(r^k - 1) \bmod p$ runs through all of $0, \dots, n-1$ as k runs through $1, \dots, n$, so the values kc_r must run through all of $\mathbb{Z}_n$, which forces $\gcd(c_r, n) = 1$. The restriction to primitive r is doing real work here, and this mode shows it: for every other r the greatest common divisor exceeds 1 and no d exists.

⟨Exhibit the perfect rows 16⟩ $\equiv$

```

for  $n := 1$ ;  $n \leq *upto$ ;  $n++$  {
   $p := n + 1$ 
  if  $\neg isPrime(p)$  {
    continue
  }
   $fmt.Printf("n=%2d \quad p=%2d \quad \phi(n)=%d \backslash n", n, p, totient(n))$ 
  ⟨Print the index table of each primitive root 17⟩
}
⟨Show that a nonprimitive  $r$  leaves no room for  $d$  18⟩

```

This code is used in section 4.

17. ⟨Print the index table of each primitive root 17⟩ $\equiv$

```

for  $R := 1$ ;  $R < p$ ;  $R++$  {
   $x := dlogRow(R, p)$ 
  if  $x \equiv \Lambda$  {
    continue
  }
   $fmt.Printf(" \quad R=%2d \quad x=\quad v \quad perfect=%v \quad all-interval=%v \quad x \sim xR=%v \backslash n",$ 
     $R, x, perfect(x), allInterval(x), equiv(x, reversed(x))$ 
  )
}

```

This code is used in section 16.

18. The table below is for the amazing row, but any perfect row tells the same story: the constant c_r is the index of r , so it is invertible modulo n exactly when r is itself a primitive root.

⟨Show that a nonprimitive r leaves no room for d 18⟩ $\equiv$

```

 $x := dlogRow(2, 13)$ 
 $fmt.Printf("\n for the row v (n=12, p=13): \backslash n", x)$ 
 $fmt.Printf("%4s %6s %6s %6s \backslash n", "r", "ord(r)", "c_r", "gcd", "")$ 
for  $r := 1$ ;  $r \leq 12$ ;  $r++$  {
   $c := (sub(x, r)[0] - x[0] + 12) \% 12$ 
   $note := ""$ 
  if  $\gcd(c, 12) \equiv 1$  {
     $note = "d \text{ exists}"$ 
  }
   $fmt.Printf("%4d %6d %6d %6d \backslash n", r, order(r, 13), c, \gcd(c, 12), note)$ 
}

```

This code is used in section 16.

19. Exhaustive verification. Nothing above proves anything; it only checks the cases the exercise names. This mode does the real work, visiting every row with $x_0 = 0$ and asking four questions of each: is it perfect, and if so does it have the all-interval property that part (a) promises, is it equivalent to its reversal as part (b) promises, and is it one of the $\varphi(n)$ tables of logarithms.

⟨ Search every row exhaustively 19 ⟩ $\equiv$

```
fmt.Printf("%3s_%3s_%12s_%8s_%8s_%6s_%7s_%6s\n",
  "n", "p", "rows", "perfect", "all-int", "x~xR", "phi(n)", "=dlog")
for n := 1; n ≤ *upto; n++ {
  p := n + 1
  if ¬isPrime(p) {
    continue
  }
  ⟨ Count the perfect rows among all (n - 1)! of them 20 ⟩
  ⟨ Report what the perfect rows satisfy 21 ⟩
}
```

This code is used in section 4.

20. ⟨ Count the perfect rows among all $(n - 1)!$ of them 20 ⟩ $\equiv$

```
var perf [][]int
total := 0
rows(n, func(x []int) {
  total++
  if perfect(x) {
    perf = append(perf, append([]int(Λ), x...))
  }
})
```

This code is used in section 19.

21. The tables of logarithms are collected into a set, and the perfect rows are looked up in it; the two agree when the counts match and every lookup succeeds.

⟨ Report what the perfect rows satisfy 21 ⟩ $\equiv$

```

logs := map[string]bool{ }
for R := 1; R < p; R++ {
  if x := dlogRow(R, p); x ≠ Λ {
    logs[fmt.Sprint(x)] = true
  }
}
ai, rev, same := 0, 0, len(logs) ≡ len(perf)
for _, x := range perf {
  if allInterval(x) {
    ai++
  }
  if equiv(x, reversed(x)) {
    rev++
  }
  if ¬logs[fmt.Sprint(x)] {
    same = false
  }
}
fmt.Printf("%3d_%3d_%12d_%8d_%8d_%6d_%7d_%6v\n",
  n, p, total, len(perf), ai, rev, totient(n), same)

```

This code is used in section 19.

22. All-interval rows, by dancing cells. Exercise 103 asks for the all-interval rows themselves, and its answer gives an XCC formulation to hand to Algorithm C. It is worth building here for two reasons: it puts a published count next to a run of our own solver, and every perfect row must appear among the solutions if part (a) is right.

The items are a position j and a pitch p_t for $0 \leq j, t < n$, an interval value d_k and an interval slot q_t for $1 \leq k, t < n$, all primary; and a secondary item x_j per position, whose color is the pitch that lands there.

⟨ Count all-interval rows with dancing cells 22 ⟩ $\equiv$

```
fmt.Printf("%3s_%8s_%9s_%11s_%10s_%s\n",
  "n", "items", "options", "solutions", "nodes", "answer_103(b)")
knuth := map[int]int{2: 1, 4: 2, 6: 4, 8: 24, 10: 288, 12: 3856}
for n := 2; n ≤ *upto; n += 2 {
  ⟨ Write the XCC problem for all-interval rows 23 ⟩
  ⟨ Solve it and compare with the published count 26 ⟩
}
```

This code is used in section 4.

23. One option per position and pitch says “pitch t goes in position j ”. Answer 103(a) shows that an all-interval row must have $x_{n-1} = (x_0 + n/2) \bmod n$, so with x_0 pinned to zero the last pitch is forced; both facts are imposed by leaving options out.

⟨ Write the XCC problem for all-interval rows 23 ⟩ $\equiv$

```
var b strings.Builder
for j := 0; j < n; j++ {
  fmt.Fprintf(&b, "j%d", j)
}
for t := 0; t < n; t++ {
  fmt.Fprintf(&b, "p%d", t)
}
for k := 1; k < n; k++ {
  fmt.Fprintf(&b, "d%d", k)
}
for t := 1; t < n; t++ {
  fmt.Fprintf(&b, "q%d", t)
}
b.WriteString("|")
for j := 0; j < n; j++ {
  fmt.Fprintf(&b, "x%d", j)
}
b.WriteString("\n")
```

This code is also defined in sections 24, 25.

This code is used in section 22.

24. $\langle$ Write the XCC problem for all-interval rows 23 $\rangle + \equiv$

```

for  $j := 0; j < n; j++$  {
  for  $t := 0; t < n; t++$  {
    if  $(j \equiv 0 \wedge t \neq 0) \vee (j \equiv n - 1 \wedge t \neq n/2)$  {
      continue
    }
     $fmt.Fprintf(\&b, "j\%d\_p\%d\_x\%d:\%d\backslash n", j, t, j, t)$ 
  }
}

```

25. The other options say “interval k sits in slot t ”, and they carry the arithmetic in their colors: the pitch at position $t - 1$ is i and the pitch at position t is $i + k$. Two options that disagree about a position cannot both be chosen, because a secondary item admits only one color, and that single mechanism is what makes the row and its intervals permutations at the same time.

$\langle$ Write the XCC problem for all-interval rows 23 $\rangle + \equiv$

```

for  $k := 1; k < n; k++$  {
  for  $t := 1; t < n; t++$  {
    for  $i := 0; i < n; i++$  {
       $fmt.Fprintf(\&b, "d\%d\_q\%d\_x\%d:\%d\_x\%d:\%d\backslash n",$ 
         $k, t, t - 1, i, t, (i + k) \% n)$ 
    }
  }
}

```

26. $\langle$ Solve it and compare with the published count 26 $\rangle \equiv$

```

 $in := b.String()$ 
 $xc := cells.NewXCC()$ 
 $res := xc.Dance(strings.NewReader(in))$ 
 $count := 0$ 
for range  $res.Solutions$  {
   $count++$ 
}
 $fmt.Printf("%3d\_8d\_9d\_11d\_10d\_10d\_10d\_10d\_s\backslash n",$ 
   $n, 5 * n - 2, strings.Count(in, "\backslash n") - 1, count, xc.Nodes(),$ 
   $knuth[n], \text{map}[\text{bool}]\text{string}\{\text{true}: "agrees", \text{false}: "DISAGREES"\}[count \equiv knuth[n]])$ 

```

This code is used in section 22.

27. Index.

a: [7](#), [9](#), [14](#).
ai: [21](#).
allInterval: [8](#), [13](#), [17](#), [21](#).
amazing: [13](#).
append: [20](#).
b: [7](#), [9](#), [14](#), [23](#), [24](#), [25](#), [26](#).
bad: [15](#).
bool: [7](#), [8](#), [9](#), [11](#), [14](#), [21](#), [26](#).
Builder: [23](#).
c: [9](#), [18](#).
cells: [4](#), [26](#).
Count: [26](#).
count: [26](#).
d: [7](#), [8](#), [9](#).
Dance: [26](#).
dlogRow: [10](#), [13](#), [17](#), [18](#), [21](#).
e: [9](#), [10](#).
equiv: [7](#), [8](#), [13](#), [17](#), [21](#).
equiv2: [12](#), [13](#), [14](#), [15](#).
Fatalf: [4](#).
flag: [5](#).
fmt: [12](#), [13](#), [15](#), [16](#), [17](#), [18](#), [19](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#).
Fprintf: [23](#), [24](#), [25](#).
gcd: [9](#), [18](#).
got: [12](#).
i: [7](#), [11](#), [14](#), [25](#).
in: [26](#).
Int: [5](#).
int: [6](#), [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#), [14](#), [15](#), [20](#), [22](#).
isPrime: [9](#), [15](#), [16](#), [19](#).
j: [6](#), [12](#), [23](#), [24](#).
k: [8](#), [9](#), [23](#), [25](#).
knuth: [22](#), [26](#).
 Knuth, Donald Ervin: [1](#).
len: [6](#), [7](#), [8](#), [14](#), [21](#).
log: [4](#).
logs: [21](#).
m: [9](#).
main: [4](#).
make: [6](#), [7](#), [8](#), [10](#), [11](#), [12](#).
mode: [4](#), [5](#).
n: [6](#), [7](#), [8](#), [9](#), [10](#), [11](#), [15](#), [16](#), [19](#), [20](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#).
NewReader: [26](#).
NewXCC: [26](#).
Nodes: [26](#).
note: [18](#).
order: [9](#), [10](#), [18](#).
out: [6](#), [7](#).
p: [6](#), [9](#), [10](#), [16](#), [17](#), [19](#), [21](#).
Parse: [5](#).
perf: [20](#), [21](#).
perfect: [8](#), [13](#), [17](#), [20](#).
Printf: [12](#), [13](#), [15](#), [16](#), [17](#), [18](#), [19](#), [21](#), [22](#), [26](#).
Println: [12](#), [15](#).
R: [10](#), [17](#), [21](#).
r: [6](#), [8](#), [9](#), [18](#).
rec: [11](#).
res: [26](#).
rev: [21](#).
reversed: [7](#), [13](#), [15](#), [17](#), [21](#).
rows: [11](#), [15](#), [20](#).
same: [21](#).
 Schoenberg, Arnold: [1](#).
seen: [8](#).
Solutions: [26](#).
Sprint: [21](#).
String: [5](#), [26](#).
string: [21](#), [26](#).
strings: [23](#), [26](#).
sub: [6](#), [8](#), [15](#), [18](#).
t: [23](#), [24](#), [25](#).
total: [20](#), [21](#).
totient: [9](#), [16](#), [21](#).
upto: [5](#), [16](#), [19](#), [22](#).
used: [11](#).
v: [10](#), [11](#).
visit: [11](#).
w: [9](#).
want: [12](#).
WriteString: [23](#).
x: [6](#), [7](#), [8](#), [10](#), [11](#), [15](#), [17](#), [18](#), [20](#), [21](#).
xc: [26](#).

- ⟨ Check what the exercise prints [12](#) ⟩ Used in section [4](#)
- ⟨ Count all-interval rows with dancing cells [22](#) ⟩ Used in section [4](#)
- ⟨ Count the perfect rows among all $(n - 1)!$ of them [20](#) ⟩ Used in section [19](#)
- ⟨ Exhibit the perfect rows [16](#) ⟩ Used in section [4](#)
- ⟨ Functions [6](#) ⟩ Used in section [4](#)
- ⟨ Print the index table of each primitive root [17](#) ⟩ Used in section [16](#)
- ⟨ Read the command line [5](#) ⟩ Used in section [4](#)
- ⟨ Report what the perfect rows satisfy [21](#) ⟩ Used in section [19](#)
- ⟨ Search every row exhaustively [19](#) ⟩ Used in section [4](#)
- ⟨ Show that a nonprimitive r leaves no room for d [18](#) ⟩ Used in section [16](#)
- ⟨ Solve it and compare with the published count [26](#) ⟩ Used in section [22](#)
- ⟨ Write the XCC problem for all-interval rows [23](#) ⟩ Used in section [22](#)

Every rth Element

	Section	Page
Introduction	1	1
Rows and their transformations	6	3
What the exercise prints	12	7
Perfect rows are tables of logarithms	16	9
Exhaustive verification	19	10
All-interval rows, by dancing cells	22	12
Index	27	14